

Assignment 1 TDT4195 Fall26

Task 1: Drawing your first triangle

1c.) Screenshot of 5 triangles drawn



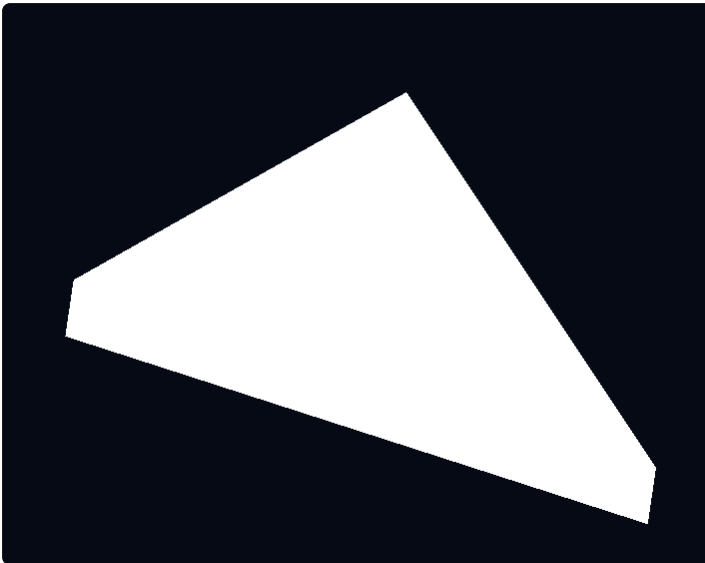
The vertices and indices used:

```
let vertices: Vec<f32> = vec![  
    0.5, -0.6, 0.0,  0.9, -0.8, 0.0,  0.7, -0.4, 0.0,  
    -0.9,  0.5, 0.0, -0.5,  0.0, 0.0, -0.7,  0.4, 0.0,  
    0.5,  0.0, 0.0,  0.9,  0.2, 0.0,  0.7,  0.4, 0.0,  
    -0.2,  0.3, 0.0,  0.3,  0.2, 0.0,  0.0,  0.8, 0.0,  
];  
let indices: Vec<u32> = vec![  
    0, 1, 2,  
    3, 4, 5,  
    6, 7, 8,
```

```
    9, 10, 11,  
    12, 13, 14,  
];
```

Task 2: Geometry and Theory

2a.)



i) What is the name of the phenomenon?

Clipping.

ii) When does it occur?

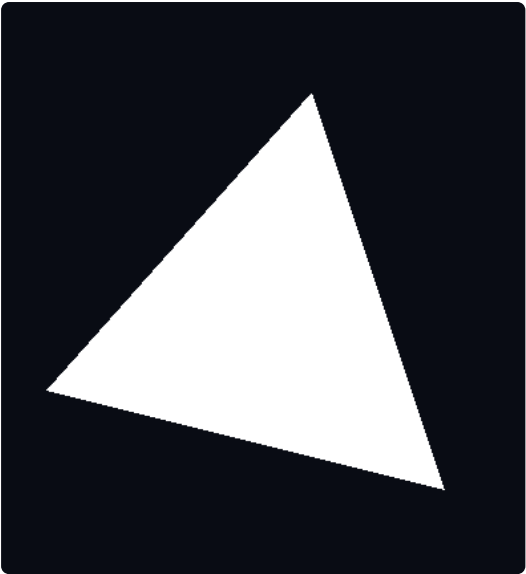
When points are placed outside of the clipping volume/view frustum. View frustum is ... This is determined by the shaders, in this case set to 1 through the `gl_Position`. In this case cause of the z values $-1, 2$ and $1, 2$ not being in the range of $[-1, 1]$.

iii) What is its purpose?

Clipping is used to ensure correct projection and for efficiency. Without clipping, elements that are behind the observer could still be projected onto the frame, causing incorrect projections.

It helps with efficiency as it prevents parts of objects that are not within the view frustum from being processed further.

2b.)

Original	Swapped
 Indices: [0, 1, 2]	 Indices: [0, 2, 1]

The vertices used were:

```
let vertices: Vec<f32> = vec![
    0.6, -0.8, 0.2,
    0.0, 0.4, 0.0,
    -0.8, -0.2, 0.2
];
```

i.) What happens?

As seen in the images above, when swapping order of the indexes nothing appears.

ii.) Why does it happen?

This is the result of back-face culling where polygons that are facing backwards are culled, and hence not rendered. The front and back side of the polygon is determined by the winding. The default order [0, 1, 2] was counter-clockwise and considered the front. By changing the order, the triangle was winded clockwise instead, causing OpenGL to think it was the back-face of the polygon and culled it.

iii.) What is the condition under which this effect occurs. Determine a rule.

Default OpenGL settings considered triangles that are winded counter-clockwise from the camera to be front-face, while counter-clockwise triangles are determined back-face and culled. Back-face culling occurs has to be enabled, using `gl::Enable(gl::CULL_FACE)`, and will then occur if the winding of the triangle is counter-clockwise from the camera's view.

2c.)

i.) Why does the depth buffer need to be reset each frame? Describe what you would observe in a scene with a sphere moving rightward when not clearing the depth buffer.

The depth buffer compares each fragment to the current value in the depth buffer to determine the depth of the fragment. Each frame the screen is cleared, and if the buffer was not reset as well, it would compare the new values to the value from the previous frame instead. This could effect the depth testing where fragments further away from the camera appearing behind other fragments fail the test and are not rendered.

If the sphere moves right, depending on how fast it moves, some of the new fragments of the sphere could be compared to fragments from previous scenes and result in an incorrect depth test. Scenes containing overlapping fragments could get a false pass or fail of the test, which could cause incorrect rendering of the sphere like fragments disappearing or leaving a trail of fragments behind.

ii.) In which situation can the Fragment Shader be executed multiple times for the same pixel? (Assume we do not use multisampling.)

If there are geometric overlap the fragment shader can be executed multiple times for the same pixel. This is because the fragment shader is run for each fragment, with each fragment as a potential pixel. In cases with overlap, multiple fragments map to the same pixel, but the fragment will be discarded if it is not seen by the camera due to overlap. Still the fragment shader executes once per fragment, even if it does not end up being a pixel, meaning it executed multiple times for that pixel.

iii.) What are the two most commonly used types of Shaders? What are the responsibilities of each of them?

The vertex shader and fragment shader are the most commonly used shaders. The vertex shader is responsible for transforming all the vertices in the scene and to project the scene onto the camera. It runs once per vertex. In comparison, the fragment shader is run once per fragment and is decides the color of the fragment.

iv.) Why is it common to use an index buffer to specify which vertices should be connected into triangles, as opposed to relying on the order in which the vertices are specified in the vertex buffer(s)?

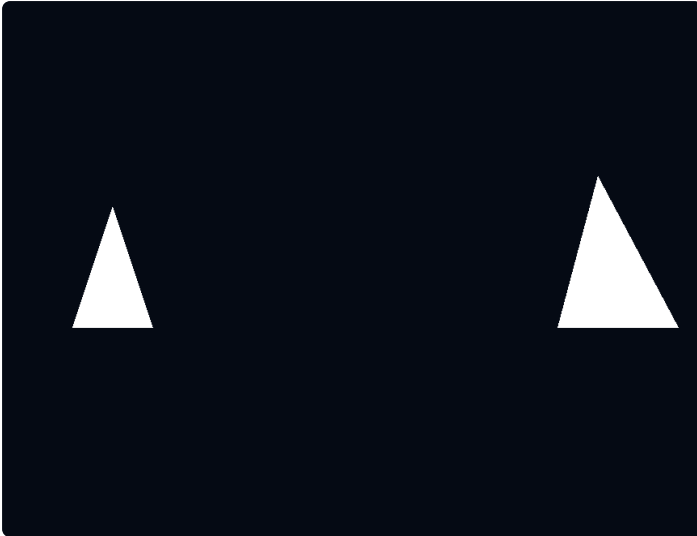
Using an index buffer allows for reuse of vertices by repeating the index, instead of having the same vertex appear multiple times in the buffer. This is important as vertices also can have attributes, which would also have to be stored double up and require more resources.

v.) While the last input of `gl::VertexAttribPointer()` is a pointer, we usually pass in a null pointer. Describe a situation in which you would pass a non-zero value into this function.

The pointer tells OpenGL the byte offset from the beginning of where the vertex attribute starts. Hence, if the vertices have attributes you do not want to include you would pass the number of bytes equivalent to the attribute you want to skip.

2d.)

Original



Used the following vertices and indices:

```
let vertices: Vec<f32> = vec![
    -0.7, -0.2, 0.0,  -0.5, -0.2, 0.0,  -0.6,  0.2, 0.0,
    0.5, -0.2, 0.0,  0.8, -0.2, 0.0,  0.6,  0.3, 0.0,
];

let indices: Vec<u32> = vec![
    0, 1, 2,
    3, 4, 5,
];
```

Mirror and flip



Altered the position in the vertex shader. Inverted the x-position to mirror, and inverted the y

position to flip.

```
gl_Position = vec4(-position.x, -position.y, position.z, 1.0);
```

Change color

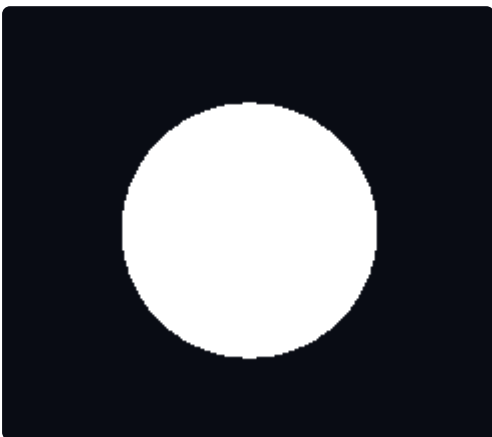


Changed color by changing the color vector in the fragment shader to as show below.

```
color = vec4(0.5f, 0.5f, 1.0f, 1.0f);
```

Task 3: Optional

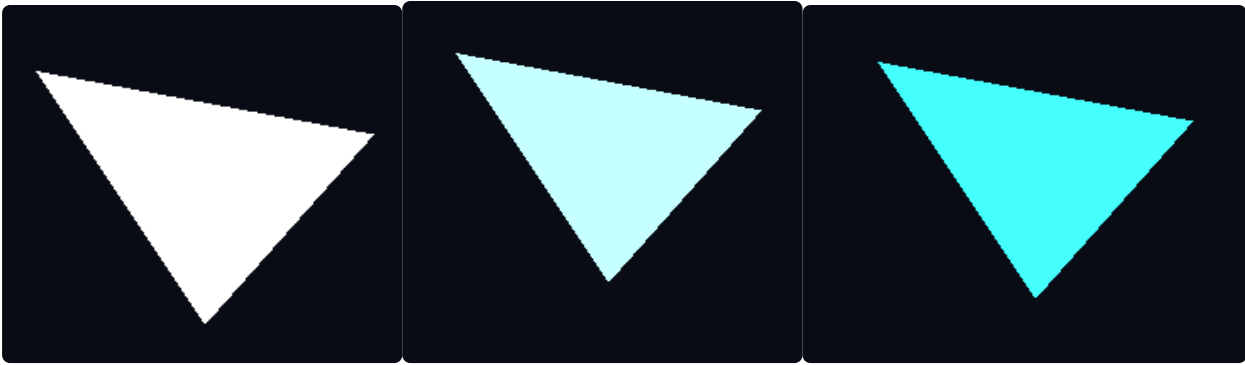
3b.) Draw a circle



b) Drew a circle. Made a function to generate vertices for a circle and drew it using `gl::TRIANGLE_FAN`. The function uses

$x = x_0 + r \cos(\theta)$, $y = y_0 + r \sin(\theta)$ to generate the vertices along the x, y plane. At first it appeared squashed and more like an oval as the window is 800x600 and squashes it, so did the simple fix and resized window to be more square.

3d.) Have an object change color over time



Used a uniform variable for time in the fragment shader. A value for the color was then calculated using $(\sin(\text{time} * 0.5) + 1)/2$. Using $\sin$ allowed for a smoother change in the interval $[-1, 1]$, which had to be modified in order to get a value in the interval $[0, 1]$. 0.5 in $\sin(\text{time} * 0.5)$ was used to make a smaller step so that the transformation would be slower.

The shader code ended up as shown below:

```
#version 430 core

out vec4 color;
uniform float time;

void main()
{
    float r = (sin(time * 0.5) + 1.0) / 2.0;
    color = vec4(r, 1.0f, 1.0f, 1.0f);
}
```

Then in `main.rs` I used `gl::GetUniformLocation` to get the location of time from the shader so that it could be changed in the main loop. In the main loop we tell OpenGL to use the `simple_shader`, and then to feed the time elapsed to the uniform location of time.

```
gl::UseProgram(simple_shader.program_id);
gl::Uniform1f(time_location, elapsed);
```

The triangle draw here used the following:

```
let vertices: Vec<f32> = vec![
    -0.9, -0.7, 0.0,  -0.5, -0.8, 0.0,  -0.7, -0.4, 0.0,
],
let indices: Vec<u32> = vec![0, 1, 2,]
```